Deusto
Facultad de Ingeniería
Ingeniaritza Fakultatea

Ecoembes Project Description

The main objective of this project is to work on the following course competences:

- GC: Teamworking
- SC1: Design software solutions, exploring different alternatives and identifying suitable patterns, accordingly.
- SC2: Document software designs using UML diagrams.
- SC3: Use and implementation of software patterns.
- SC4: Produce a distributed application in Java, implementing an object-relations mapping for data management.

To do this, the starting point is this brief initial description that will be rounded off by each team as the project progresses. We intend to build a distributed system that implements a reduced version of a **packaging recycling solution**. The development process will be incremental and 3 prototypes will be released:

- **Instructions for Submission 1 - Prototype TW1 (10% final grade)**: Initial UML class diagram and one sequence diagram at component level (to illustrate the API behaviour), the first version of the central server and server API validation, by means of http requests from a browser or using Postman).
- **Instructions for Submission 2 - Prototype TW2 (20% final grade)**: UML class diagram (updated) and detailed sequence diagram (adding persistence) and patterns for communication with external servers (external recycling plants), updating the central server and the implementation of the recycling plant services. The new prototype must be fully functional.
- **Instructions for Submission 3 - Prototype TW3 (14% final grade)**: UML class diagram (final version) and one sequence diagram (integration of the client side), GUI development using Java Swing or a web client based on Thymeleaf. The final prototype must be fully functional.

The 3 prototypes will be developed in an incremental way: each prototype will include the previous one, refining and correcting possible faults, errors and mistakes. Additionally, all the process will be under a continuous assessment process based on in-person formal technical reviews:

Application Domain Description

Ecoembes is an association that aggregates and represents organizations and companies linked to packaging recycling. Its aim is to deploy a smart solution that engages all stakeholders

involved in recycling, from citizens to recycling plants, including the management of kerbside recycling dumpsters and garbage trucks.

Kerbside dumpsters have a special device (a Recyclobin Ultraeye sensor ®) that provides an estimate of the dumpster's fill level and transmits that information daily to the central server.

Dumpster information (location and fill level) is used as input to a route generator, which returns a list of optimal routes for dumpster emptying. Routes are assigned to garbage trucks.

Trucks have their own smart tablet to display routes, as well as the estimated duration each section of the route takes. Recycling plants are notified daily indicating the number of dumpsters and number of packaging waste they will receive, so that they can plan their daily work.



Kerbside recycling dumpster

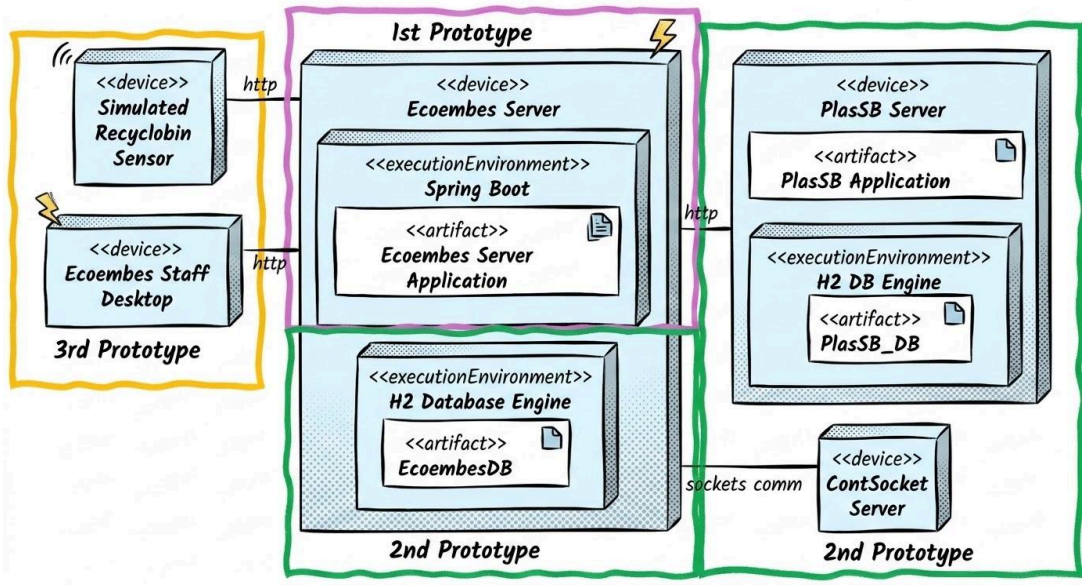
Currently, two recycling plants are engaged in this solution: *PlasSB Ltd*. and *ContSocket Ltd*. For any given date, both plants will provide their recycling capacity in tons (this data is always available 10 days in advance). Attention: the solution should be designed to accommodate more recycling plants in a flexible and transparent way.

Ecoembes staff will have a desktop application for their dumpster, truck and recycling plant management tasks, as well as a dashboard for statistics.

Employees will have access to several key functions within the system. First, they can download up-to-date information about the dumpsters, including their locations, the number of containers they hold, and their current fill levels. Next, employees can select any recycling plant and check its available capacity (in tons) for the current day. Based on this information, they can then choose one or more dumpsters and assign them to the selected recycling plant for processing. Once the assignment is completed, the chosen recycling plant automatically receives a notification detailing the total number of dumpsters being sent and the combined number of containers they contain. The identity of the employee who performed the assignment will be recorded for auditing purposes.

All information about dumpsters, recycling plants and employee details is stored in a repository hosted on the central server.

Deployment diagram (3 prototypes)



The initial and basic functionality that the system to be developed must include is described below:

Dumpsters

- **Update dumpster info**: Each dumpsters will have a unique identifier and a location (address); the dumpster sensor will transmit daily updates to the central server at 3 AM (to be simulated in the prototypes), including the dumpster id, the estimated number of containers and fill level. The fill levels are classified into three statuses: green (there is room for more containers), orange (limited space) and red (indicating dumpster full). This information will be stored in the centrar server's database.

Employee Functions

- **Login**: Using the email address of the employees and password. When login, a *token* is generated (for simplicity we will use the timestamp of the moment the login process is processed on the server side) that is returned to the client side and it is used as a parameter in the rest of the interactions with the server. This token guarantees that the login has been performed before executing any other functionality. The server must store "in memory" the list of tokens and employees' data who are logged in, in order to minimize accesses to the database. This information represents the concept of server "state" and its goal is to reduce the amount of information exchanged between the server and the client.
- **Logout**: the logout process basically consists of deleting a token (and the rest of the associated information). From the moment the logout process is carried out, the

employee can no longer request any other functionality, unless the login process is executed again.

- **Create a New Dumpster**: Employees can add new dumpsters to the system, providing details such as a dumpster identifier, its location and initial capacity. (Optional: the employee who adds a new dumpster gets a confirmation email).

- **Query Dumpster Usage**: Employees can check how a specific dumpster has been used over a period of time, between a chosen start date and end date. This helps track trends and monitor usage patterns.

- **Check Dumpster Status**: Employees can see the status of dumpsters in a particular area (identified by postal code) on a specific day. This provides a snapshot of dumpster activity in different locations.

- **Check Recycling Plant Capacity**: Employees can view the available capacity at recycling plants for a given date, ensuring that packaging containers are only sent where there is enough space to process them.

- **Assign Dumpsters to Recycling Plants**: The system allows employees to allocate dumpsters to the appropriate recycling plants based on the total number of packaging containers and the plant's capacity. The plant is notified of the expected number of dumpsters and packages, and the system keeps a record of which employee performed the assignment.

- **(Optional) Saturation Indicator**: If more than 80% of dumpsters reach a high fill level ("orange"), the system automatically sends an email alert to all employees. This helps prevent overfilled containers and ensures timely collection.

Deusto
Facultad de Ingeniería
Ingeniaritza Fakultatea

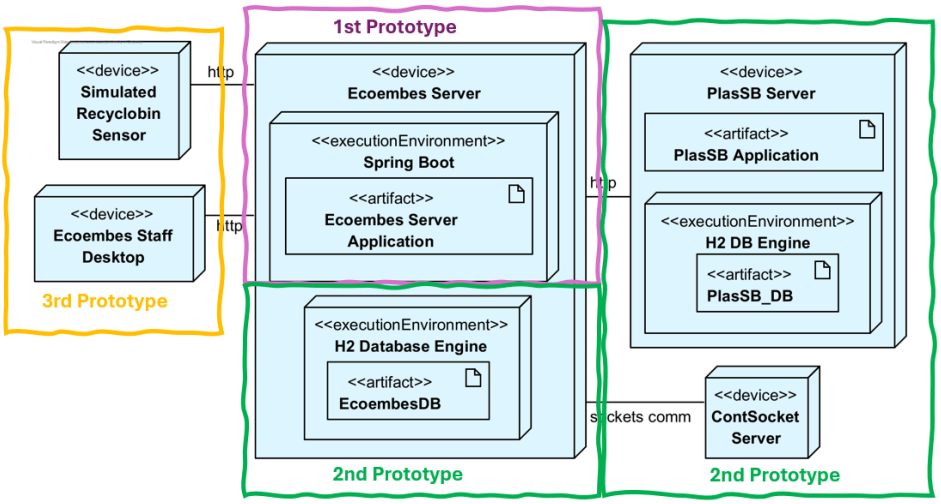
Description - Prototype Ecoembes TW1 - (10% of the grade)

Personal work - time estimation: 20h./person

The aim of this prototype is the design, implementation and validation of the **Central Server API**. To do this, the following tasks will have to be carried out:

1. **Designing the Central Server API**, including all the functionality described on the project description document. For each operation, you must specify the type (GET, POST, etc.), the name, and the input and output parameters (including their types/objects).
2. **Modeling of the solution's first class diagram,** defining the classes that configure the foundational architecture of the Central Server, and incorporating the Façade, AppService, and DTO patterns. Additionally, you must define the information structure that the Central Server will use for state management (StateManagement pattern) for the various requests that will come from the client side.
3. **Modeling of a Sequence Diagram at component level, representing an overview of all the functionality.** This diagram must only represent the client side, the Central Server and the recycling plants external servers: *PlasSB* server and *ContSocket* server.
4. **Implementation of the first prototype of the Central Server,** by means of a Java project using Spring Boot and Gradle. This first prototype will NOT contain persistence, and all functionality will be simulated in the service layer. The code must be fully functional and error-free. The API must be documented with Swagger.
5. **Validation of the Ecoembes TW1 Prototype** through invocations from a browser using Swagger or Postman. The submission will include screenshots that provide evidence of the proper functioning of each API operation.

Deployment Diagram (1st prototype)



Deusto
Facultad de Ingeniería
Ingeniaritza Fakultatea

CHECKLIST BEFORE SUBMISSION - ASSESSMENT POINTS

- Central Server API: complete; methods, arguments and return types (REST style).
- Central Server API: consistent with methods in façade in class diagram.
- Class Diagram: complete, all classes; all patterns clearly modelled. Team decisions are clear.
- Sequence Diagram: component level; client-side represented; external services represented; remote messages in sequence diagram consistent with Central Server API, and external servers' services.
- Prototype: consistency between diagrams and code; proper package structure; simulation code complete and fully functional.

SUBMISSION FORMAT

The submission is made up of 2 parts:

- One document that includes the following sections:
    1. The detailed description of the Central Server API, in a table format that outlines each function, including its name, type, input parameters, and output parameters.
    2. Image (s) of the Class Diagram
    3. Image (s) of the Sequence Diagram
    4. Screenshots that demonstrate the correct functioning of each operation of the Ecoembes Server.
- Folders with the structure and source code of the 1st version of Ecoembes Server

Submission on Alud

One compressed file, named with the name of the team. Maximum file size: 50 MB.

TECHNICAL REVIEW - PROTOCOL (30 minutes)

Sequence of the review: Central Server API table, Class Diagram, Sequence Diagram. Demo (test cases for all functionality described in the project description document).

Roles: instructor acting as moderator/listener; one/two team member taking notes in turns (recorder of defects).

- **All team members:** explain the submission points. Members decide who is going to explain which part.
- Every member has around 5 minutes to provide explanations.

Last step - focus group (and Team Charter)

- Verbalize strengths and weaknesses
- Reflection & Qualitative self-assessment:
    - ‘work in progress’ / ‘pass’ / ‘adequate’/’very good’ / ‘excellent’
- Comments on the agreed individual contribution to the team

Ecoembes TW1 - SD E2

1.1. Central Server API V1 - Neutral design.

updateDumpsterInfo(dumpsterID: long, numContainers: int, fillLevel: FillLevel) : void

login(email: String, password: String) : long

logout(token: long) : void

createDumpster(token: long, dumpster: DumpsterDTO) : DumpsterDTO

getDumpsterUsage(token: long, dumpsterID, startDate: LocalDate, endDate: LocalDate) : List<DumpsterUsageDTO>

getDumpsterStatusByArea(token: long, postalCode: String, date: LocalDate) : List<DumpsterStatusDTO>

getPlantCapacity(token: long, plantID: long, date: LocalDate) : double

assignDumpstersToPlant(token: long, plantID: long, dumpsterIDs: List<Long>) : AssignmentDTO

getAllDumpsters(token: long) : List<DumpsterDTO>

getAllPlants(token: long) : List<PlantDTO>

1.2. Central Server API V1 - REST design.

Operation name	Type	Path	Input	Output
updateDumpsterInfo	POST	/dumpsters/{dumpsterId} /measurements	dumpsterId (long), numContainers (int), fillLevel (FillLevel)	- - -
login	POST	/sessions	email (String), password (String)	long
logout	DELETE	/sessions	token (long)	- - -
createDumpster	POST	/dumpsters	token (long), dumpster (DumpsterDTO)	DumpsterDTO
getDumpsterUsage	GET	/dumpsters/{dumpsterId}/usage	token (long), dumpsterId (long), startDate (LocalDate), endDate (LocalDate)	List<DumpsterUsageDTO>
getDumpsterStatusByArea	GET	/dumpsters/status	token (long), postalCode (String), date (LocalDate)	List<DumpsterStatusDTO>
getPlantCapacity	GET	/plants/{plantId}/capacity	token (long), plantId (long), date (LocalDate)	double
assignDumpstersToPlant	POST	/plants/{plantId}/assignments	token (long), plantId (long), dumpsterIds (List<Long>)	AssignmentDTO
getAllDumpsters	GET	/dumpsters	token (long)	List<DumpsterDTO>
getAllPlants	GET	/plants	token (long)	List<PlantDTO>

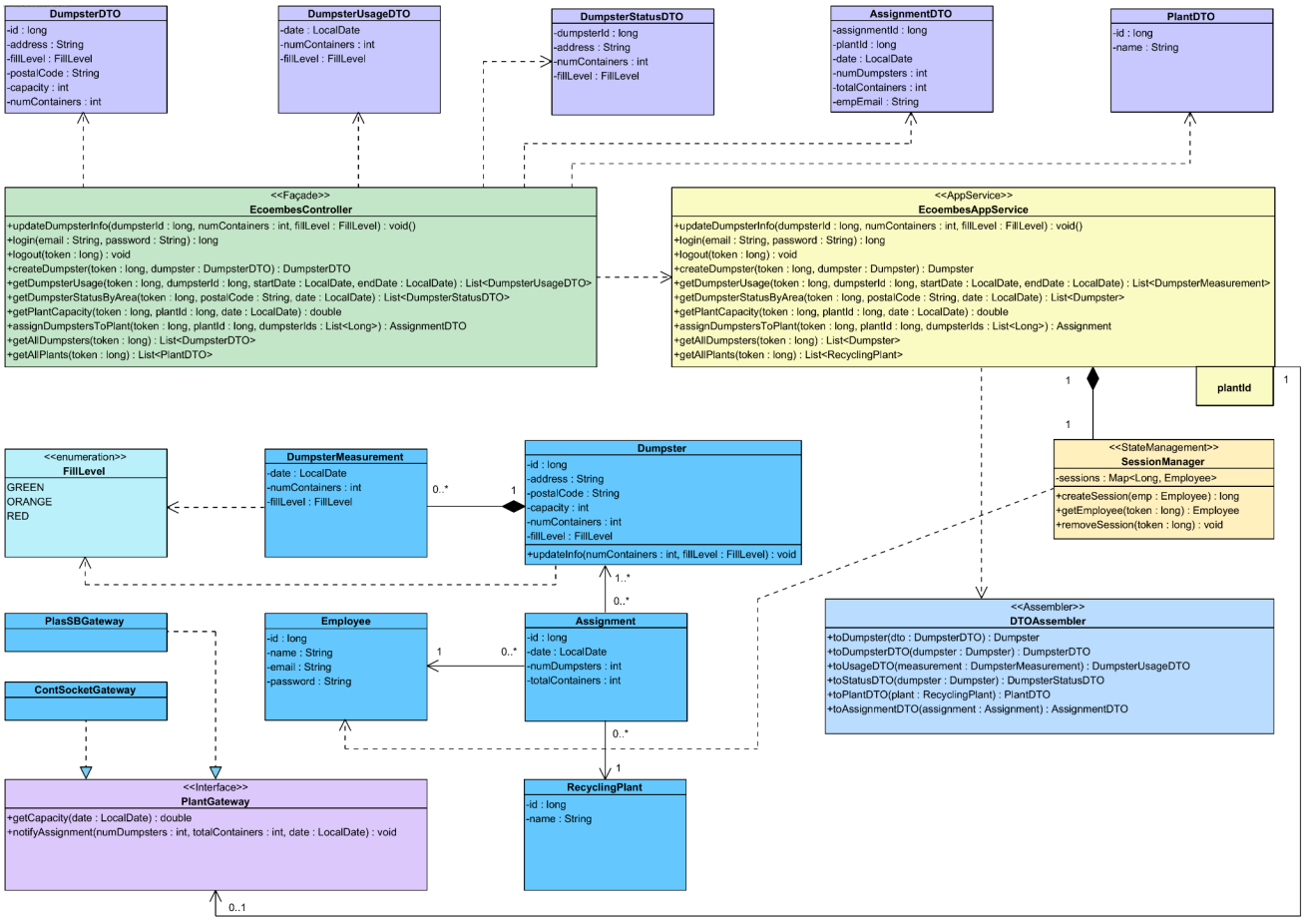
Parameter legend: **HEADER** · **PATH** · **BODY** · **QUERY**

DTO	Fields
DumpsterDTO	id (long), address (String), postalCode (String), capacity (int), numContainers (int), fillLevel (enum: GREEN-ORANGE-RED).
DumpsterUsageDTO	date (LocalDate), numContainers (int), fillLevel (enum).
DumpsterStatusDTO	dumpsterId (long), address (String), numContainers (int), fillLevel (enum).
AssignmentDTO	assignmentId (long), plantId (long), date (LocalDate), numDumpsters (int), totalContainers (int), empEmail (String).
PlantDTO	id (long), name (String).

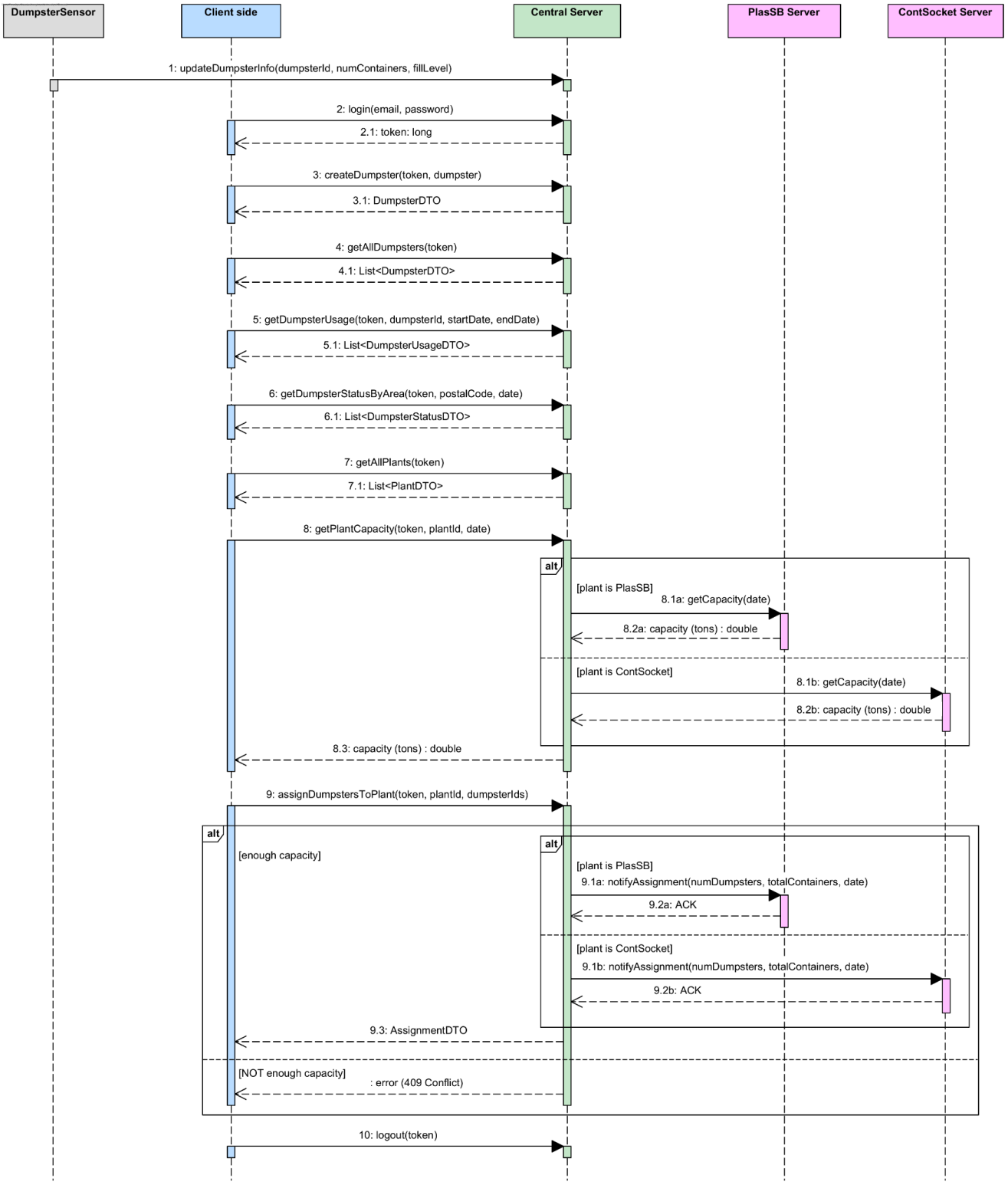
Design decisions:

- We modeled *updateDumpsterInfo* as POST because *getDumpsterUsage* requires historical data, so each daily update from the sensor is a new record to be stored.
- We modeled *login* and *logout* as POST and DELETE, respectively, on a resource named "/sessions" to represent the creation and deletion of a session, instead of introducing "/login" and "/logout", since they go against REST principles.
- We did not include the saturation indicator as an API operation because it is automatically triggered by the system. Also, the confirmation email on dumpster creation and the plant notification after an assignment are side effects of their respective operations, not endpoints.
- We defined a token as a type *long* (timestamp in milliseconds) since it can be easily transmitted as a URL path parameter.

2. Class Diagram V1.



3. Client-side Sequence Diagram.



4. Prototype validation (Swagger).

Here are the links to the screenshots of every possible response type for each operation:

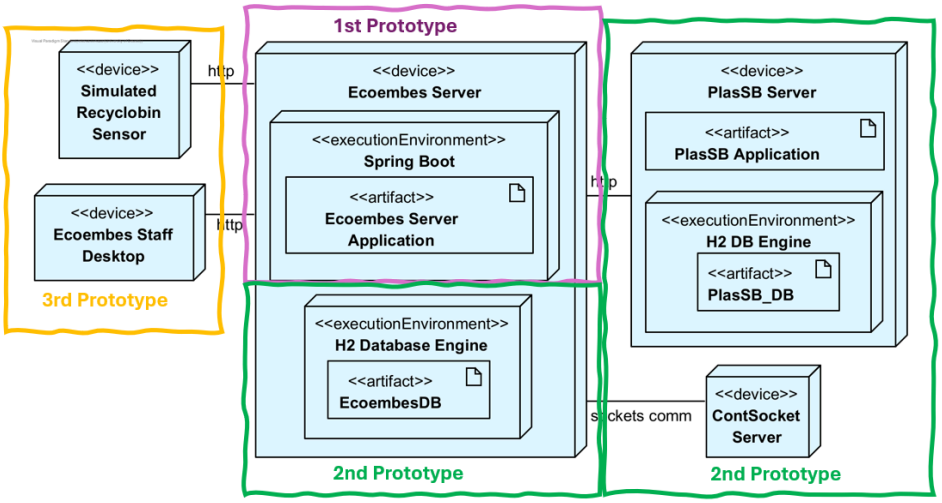
- updateDumpsterInfo
- login
- createDumpster
- getAllDumpsters
- getDumpsterUsage
- getDumpsterStatusByArea
- getAllPlants
- getPlantCapacity
- assignDumpstersToPlant
- logout

Description - Prototype Ecoembes TW.2 - (20% of the grade)

Personal work - time estimation: 20h./person

The aim of this second prototype is to integrate the interaction with the external servers *PlasSB* server and *ContSocket* server that provide their own functionality, as well as adding persistence to the Central Sever.

Deployment Diagram (2nd prototype)



To do this, the following tasks will have to be carried out:

1. **Designing the API on *PlasSB* server and *ContSocket* server.** You must provide details about the methods exposed in each recycling plant service (name of the methods, parameters and their types and the return type). The ***PlasSB* server** will be simulated as a Spring Boot server, persistence being implemented by means of JPA+H2; and the ***ContSocket* server** will be a socket-based server, and data storage will be simulated in-memory.
2. **Modeling of the solution's new class diagram,** including**:**
   - Corrections and improvements on the classes in the Prototype TW1 class diagram.
   - DAO Pattern, for persistence.
   - Service Gateway Pattern, for the communication with the recycling plant external services.
   - Factory Pattern, for creating the Service Gateway objects.
3. **Modeling of a Sequence Diagram that represents the *Check Recycling Plant Capacity* functionality**, which illustrates the interaction with the external services. The

diagram must include the complete interaction (with all **classes**) from the client side to the external services. Since there is no client side in this version, this part will be represented by an actor that interacts with the remote façade of the Central Server.

4. **Implementation of the second prototype,** adding the new classes, being the minimum requirements for the implementation of this second prototype:
    - Evolution of the Prototype TW1, to introduce the communication with the recycling plant servers, through their corresponding service proxies. In addition, persistence will have to be added.
    - The prototype will be organized in 3 projects: Ecoembes Server, PlasSB Server and ContSocket Server.
    - As an information repository, both in the Central Server and in PlasSB Server, a database management system must be used. For simplicity, we recommend you use H2.
    - The code must be organized in several packages (following the same structure as the sample case code provided as a reference).
    - The code must be free of execution errors and warnings.
5. **Validation of the Ecoembes TW2 Prototype** through invocations from the browser using Swagger or Postman. The submission will include screenshots that provide evidence of the proper functioning of each API operation.

CHECKLIST BEFORE SUBMISSION - ASSESSMENT POINTS

- API for the 2 recycling plant servers is clear.
- Class Diagram: complete, all classes; all patterns clearly modelled. Team decisions are clear.
    - Service Gateway implement one interface (according to the Service Gateway pattern specification)
    - Factory calls the 'constructors' of the target classes, and is modeled according to the Factory specification.
    - Factory used as it should be, hiding the real class the target object belongs to.
- Sequence Diagram: proper participants in the diagram, all patterns displayed on diagram.
- Prototype: consistency between diagrams and code; proper package structure; simulation code complete and fully functional (Client requests + Server patterns + Integration patterns + S2S comm. working with Spring Boot and Sockets + Persistence)
- Code double-check: RF & AppService unaware of type of external recycling plant server used. If-then-else or switch-case code only into the Factory code. Best Practice: No hardcoded IP or ports (tip: try use of property files)

SUBMISSION FORMAT

The submission is made up of 2 parts:

- One document that includes the following sections:

1. The description of the recycling plants API on the external servers *PlasSB* server and *ContSocket* server.
2. Image (s) of the Class Diagram
3. Image (s) of the Sequence Diagram of the "Check Recycling Plant Capacity" functionality
4. Screenshots that demonstrate the correct functioning of each operation of the Ecoembes Server.

- Folders with the structure and source code of the 3 projects (Ecoembes Server, *PlasSB* Server and *ContSocket* Server)

Submission on Alud

One compressed file, named with the name of the team. Maximum file size: 50 MB.

TECHNICAL REVIEW - PROTOCOL (40 minutes)

Sequence of the review: New Class Diagram, Sequence Diagram documenting "Check Recycling Plant Capacity" process, recycling plants API on the external servers, demo (test cases: all functions described in the problem description, including "Check Recycling Plant Capacity" using both external recycling plants API, and validation of the contents of the central server H2 database).

Roles: instructor acting as moderator/listener; one/two team member taking notes in turns (recorder of defects).

- **All team members:** explain the submission points. Members decide who is going to explain which part.
- Every member has around 5 minutes to provide explanations, sharing his/her screen

Last step - focus group

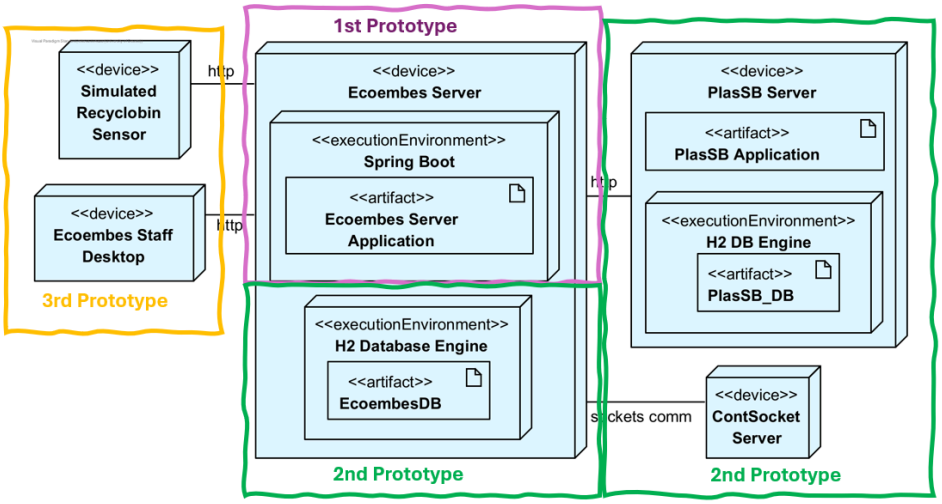
- Verbalize strengths and weaknesses
- Reflection & Qualitative self-assessment:
  - 'work in progress' / 'pass' / 'adequate'/'very good' / 'excellent'
- Comments on the agreed individual contribution to the team.

Description - Prototype Ecoembes TW.3 - (14% of the grade)

Personal Work - time estimation: 20h./person

The aim of this prototype is to review Prototype 2 and add the client side.

Deployment Diagram (3rd prototype)



To do this, the following tasks will have to be completed:

1. **Modeling of the solution's new class diagram,** adding:
   - Corrections and improvements to Protype TW.2 class diagram.
   - Classes that correspond to the client side.
2. **Modeling of a Sequence Diagram that represents the 'assignment of dumpsters to recycling plants' functionality**, which illustrates the complete set of steps starting on the client side up to the interaction with the database.
3. **Implementation of the final prototype, incorporating the client-side**. The client-side implementation can be done using a GUI based on Java Swing or a web interface with Thymeleaf.
   - The new prototype will be organized on 4 independent projects: the Ecoembes Server, PlasSB Server, ContSocket Server and the Ecoembes Client.
   - The code must be free of execution errors and warnings.

CHECKLIST BEFORE SUBMISSION - ASSESSMENT POINTS

- Class Diagram: complete, all classes; all patterns clearly modelled. Team decisions are clear.
- Sequence Diagram: proper participants in the diagram, all patterns displayed on diagram, including the client patterns.
- Prototype: consistency between diagrams and code; proper package structure; simulation code complete and fully functional (GUI/Web + Client patterns + Server patterns + Integration patterns - S2S comm. working with SB and sockets + Persistence using DAO Patterns + JPA+H2)

SUBMISSION FORMAT

The submission is made up of 2 parts:

- One document that includes the following sections:
  1. Image (s) of the Class Diagram
  2. Image (s) of the Sequence Diagram
- Folders with the structure and source code of the 4 projects.

Submission on Alud

One compressed file, named with the name of the team. Maximum file size: 50 MB. (remove the .git folder in all projects to reduce the size of the compressed file)

TECHNICAL REVIEW - PROTOCOL (30 minutes)

Sequence of the review: New Class Diagram, Sequence Diagram documenting the assignment of dumpsters to recycling plants process, demo execution using the new client side.

For the demo to be meaningful, you must preload some initial data (one or two employees, who login/logout; enough dumpster information so the queries are meaningful). Prepare the data so you can test all the requested functionality.

Demo test cases:

- After login, the employee creates a new dumpster (and it will be stored in the database). Optional - they get an informative email using JavaMail and Gmail, find sample code in ALUD and additional info).
- Using Swagger or Postman, simulation of the update of the new dumpster information.
- The employee runs dumpster queries and checks the recycling plants capacities. They will assign dumpsters to a recycling plant (after the proper validation calculations) and the proper information will be stored in the database. In addition, the recycling plant

will get a informative notification with the expected number of dumpsters and packages.
- Optional functionality.
- Logout.

Roles: instructor acting as moderator/listener; one/two team member taking notes in turns (recorder of defects).

- **All team members:** explain the submission points. Members decide who is going to explain which part.
- Every member has around 5 minutes to provide explanations

Last step - focus group

- Verbalize strengths and weaknesses
- Reflection & Qualitative self-assessment:
    - ‘work in progress’ / ‘pass’ / ‘adequate’/’very good’ / ‘excellent’
- Comments on the agreed individual contribution to the team.